

Part 4: Connect to Server

Goal: connect the LUX Lab widget to the instance through its server script, and see live data on the LUX page.

Where this sits: Part 1 Setup → Part 2 Build Experience → Part 3 Learn Lit.js → **Part 4 Connect to Server**

Steps

1. **Recap from Part 3**
   - Widget renders with default values only
2. **How widget and server talk**
   - Server script fills `data`
   - Widget reads it as `this.data`
   - Same idea as Service Portal server script
3. **The link:** `@server('./server-script.js')`
4. **Open** `widgets/aiux-lux-lab/server-script.js`
5. **The function signature**
   - `export default function server(data, _options, _input)`
   - `data`: what we send to the widget
   - `_options`, `_input`: not used today
6. **Simple values first**
   - `data.greeting`, `data.title`
   - Deploy, watch the defaults get replaced
7. **Server timestamp**
   - `new GlideDateTime().getDisplayValue()`

- `try / catch`, error into `data.timestampError`

8. **Latest incident**
   - `GlideRecordSecure('incident')`: respects ACLs
   - `orderByDesc('sys_created_on')`, `setLimit(1)`
   - Send only what the widget needs: `number`, `shortDescription`
   - Error into `data.incidentError`
9. **Back to the widget**
   - `this.data?.timestamp`
   - `this.data?.incident` → number + description
   - No incident → "No incidents found"
10. **Build, deploy, open `/lux-lab`**
11. **Test it**
    - Admin: latest incident shows
    - User without incident access: "No incidents found"
12. **End-to-end recap**
    - LUX page → LUX Lab widget → server script → `data` → `render()` → screen
13. **Checkpoint**
    - Title, greeting, timestamp and incident all from the server

Series wrap-up: Setup → Page → Widget → Server, a full AIUX/LUX experience.

AINOW
Subscribe for more ServiceNow + AI deep dives
youtube.com/@aibytusharm
Scan to subscribe

